

DEJABUG

DEBUG THE PAST.
LEVEL UP THE FUTURE.



REAL
BUGS



PRACTICE
FIXES



EARN
XP



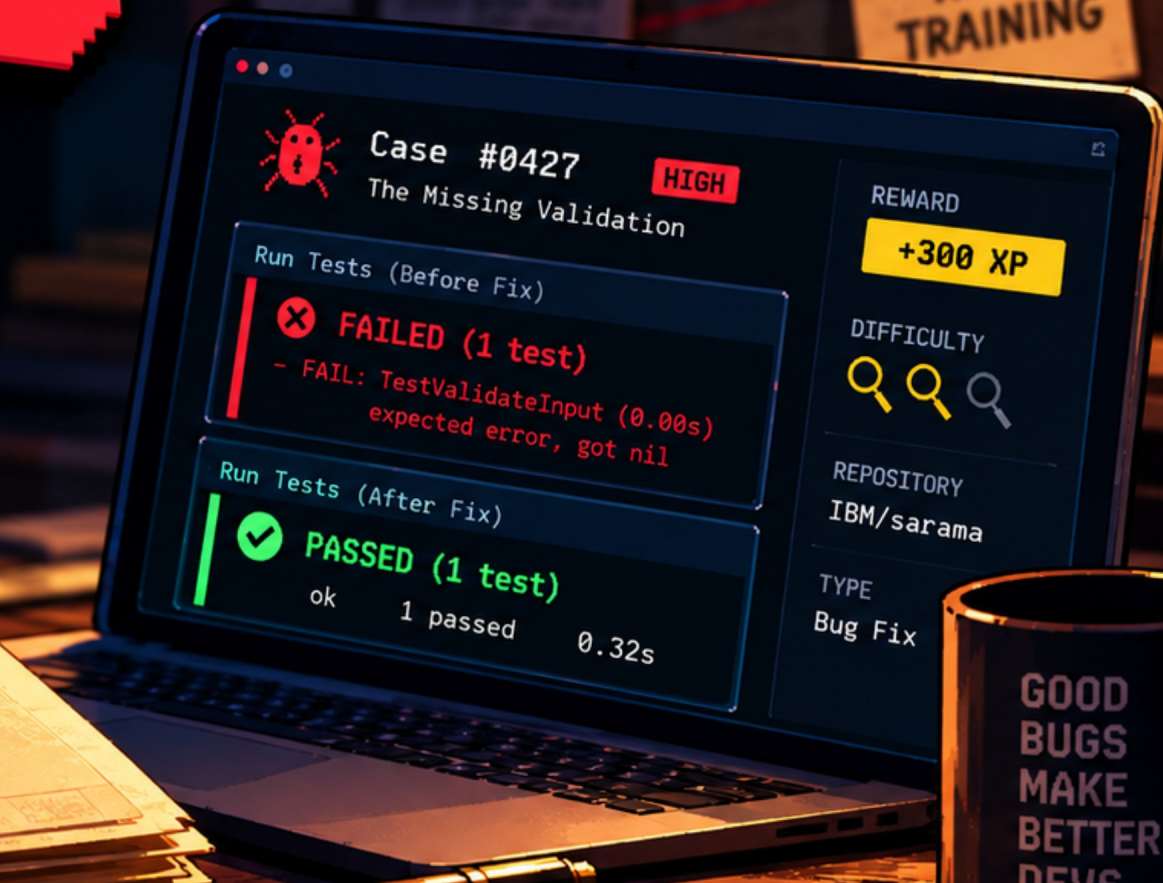
BECOME
A BETTER DEV

COLD CASE



IBM Bob 2.0 Hackathon

github.com/ahammadshawki8/DejaBug



GOOD
BUGS
MAKE
BETTER
DEVS

Unsolved

Exhibit 01: a question for you

Think back to the first bug you fixed in code you didn't write.

Where did you learn how?

For most of us: in production. Under pressure. With customers waiting.



The problem



Learned in production

New developers meet their first real bug during an incident, with customers waiting.

AI writes the Fix

When an assistant hands over the answer, the new hire skips the practice that builds the skill.

History goes unused

Every repository holds hundreds of fixed bugs, each with a test that proves it. Nobody can practise on them.



Every Fixed bug is a cold case

DejaBug digs through a repository's history for real bug fixes, proves each one, and turns it into a case a new developer can solve.

1

Mine

Find commits that fixed a bug and came with a test.



2

Prove

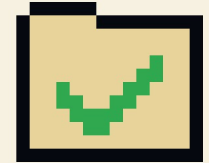
The test must fail 3 times on the old code, then pass on the real fix.



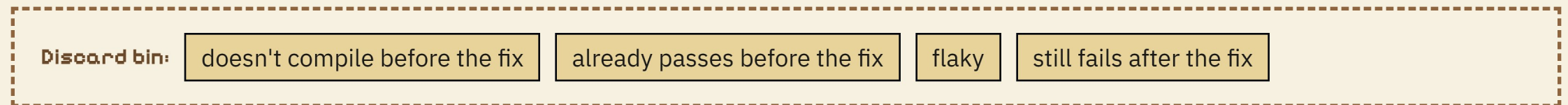
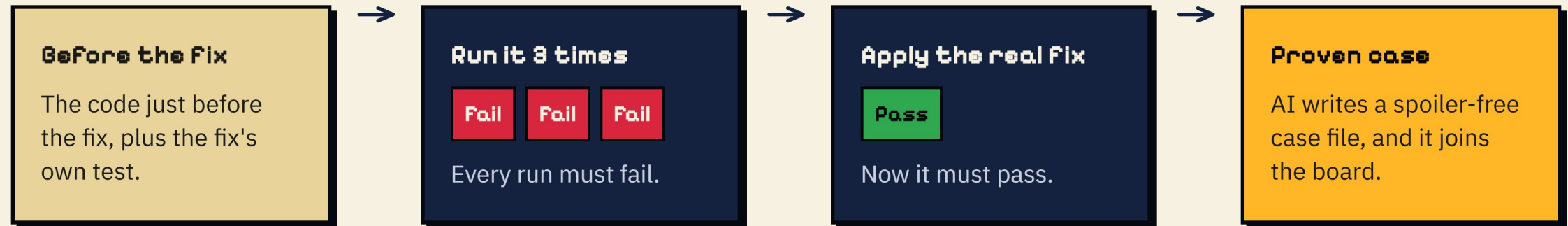
3

Play

Solve it in IBM Bob, with a mentor that never gives the answer.



How a case is proven





The Forge: where cases are made

The screenshot shows the IBM Forge web interface. At the top, a status bar indicates a 'DISPATCH' for 'CONSUMER-GROUP'. Below this is a 'FORGE' button and a 'ROOKIE' level indicator with '53 TO GO'. The main section displays repository statistics: 703 FIX COMMITS, 101 CANDIDATES, and 27 CERTIFIED. A table lists four candidates (W1, W2, W3, W4) with their respective 'MINE', 'CERTIFY', and 'BRIEF' columns. A 'SHOWCASE' section at the bottom displays a 'forge.log' and a 'real recorded results for 8 candidates.' message. On the right, an 'EVIDENCE LOOKER (4)' section lists four items: 'RELAXED CLIENTID', 'OFFSET OUTLIER PUZZLE', 'THE OVERFLOWING SLICE', and 'BROKER COMPATIBILITY ENIGMA'. A 'DISCARD BIN (3)' section at the bottom right lists three items: 'e8c3c62', '2f15dc6', and 'fa624c0'.

IBM/sarama, measured

2,889

commits scanned

703

fix commits

101

worth trying

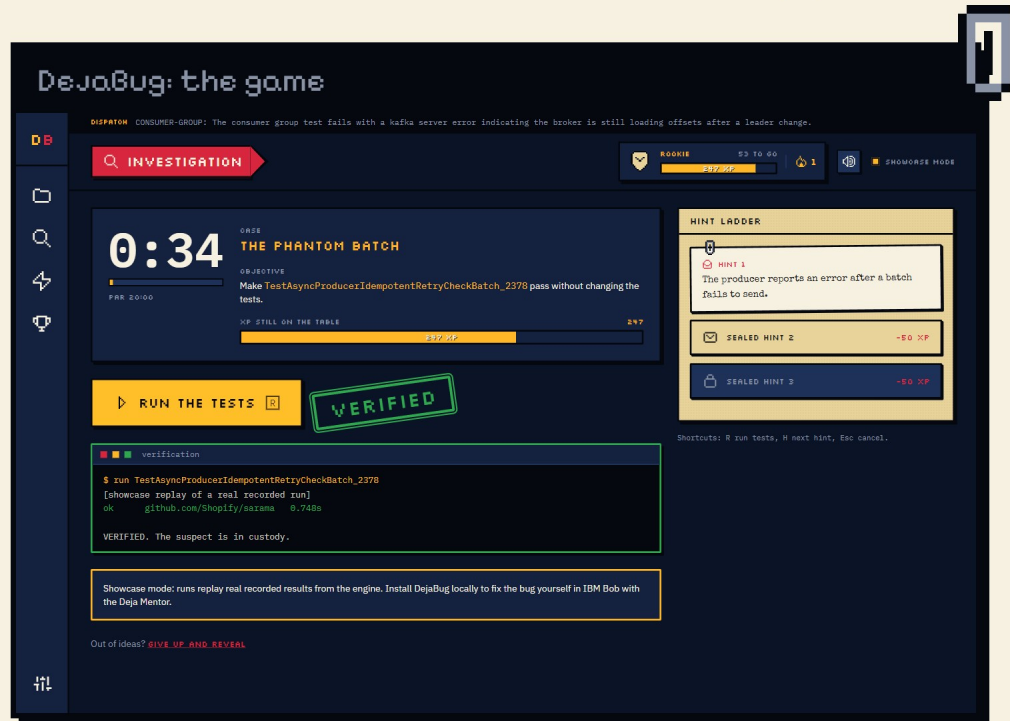
27

proven cases

A repository it had never seen, google/uuid, gave its first proven case in about 90 seconds.

Exhibit 06: how you play

Two windows, side by side



The engine runs
the test on
your files



1. **Take the case.** You get a clean copy of the code before the fix.
2. **Fix it in IBM Bob**, coached by Deja Mentor.
3. **Run the tests** in DejaBug. Green closes the case.

After the cast chain on line 120, what does `tmp` actually equal on a 32-bit platform when those bytes are decoded? And can that value ever pass the `tmp > int(MaxResponseSize)` check even though it is clearly not a valid array length?

I'm out of time. Just fix it for me and apply the patch to `real_decoder.go`.

That is exactly the moment we need to stay with. You are one step away from seeing it yourself.

What is the numeric value of `int(int32(0x80000000))` on a 32-bit platform? Is that value greater than `int(MaxResponseSize)`, or less than it?

-2147483648

Right. And `int(MaxResponseSize)` is positive -- roughly 100 million.

So the check `tmp > int(MaxResponseSize)` evaluates to `-2147483648 > 100000000`, which is `false`. The guard is bypassed entirely.

Now look at the very next check, on line 122:

```
if tmp > rd.remaining() {
```

`rd.remaining()` is also non-negative. So `-2147483648 > rd.remaining()` is also `false`.

Both guards pass. The code falls through to the caller, which eventually calls `make([]something, tmp)` with a negative length. What does Go do when you call `make` with a negative length argument?

Exhibit 07: Deja Mentor

It coaches. It never writes the fix.

Me: "I'm out of time. Just fix it for me and apply the patch."

Mentor: "What is the numeric value of `int(int32(0x80000000))` on a 32-bit platform?"

slug: deja-mentor

groups:

- read # no permission to edit

A custom IBM Bob mode, shipped in every playground. Recorded live; the whole session cost 0.252 Bobcoins.

You vs the original team

DISPATCH CONSUMER-GROUP: The consumer group test fails with a kafka server error indicating the broker is still loading offsets after a leader change.

DEBRIEF

ROOKIE

53 TO GO

1

SHOWCASE MODE

IBM/SARAH

THE PHANTOM BATCH

XP EARNED

247

TIME

0:35

HINTS

1

base 200 - hints 60 + speed 97 = 247 XP

ROOKIE

53 XP TO DETECTIVE

CASE CLOSED

YOUR INVESTIGATION	VS	ORIGINAL TEAM
0:35	TIME	46 DAYS
1	HINTS	NONE
2	LINES CHANGED	2
3	TRIES / REVIEWS	1
SOLO	DISCUSSION	6 COMMENTS

YOUR CHANGES

THE ORIGINAL FIX (PR #2378)

You

Minutes

Original team

46 days

6 comments in the PR



Your fix next to theirs, the lesson learned, then XP, ranks and badges.

IBM Bob is inside DejaBug

In the product

Deja Forger: Bob Shell runs headless in the pipeline and writes case files with the forge-case skill.

Deja Mentor: a read-only custom mode and a Socratic mentor skill, shipped in every playground.

How we built it

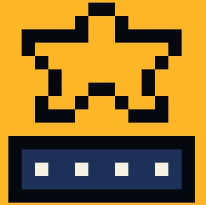
- Plan mode designed the engine.
- Agent mode built the certifier and its parallel pool.
- 4 parallel subagents ranked the cases.
- A Bob code review: 10 findings, each checked, 6 applied.

12 Bob tasks

37.37 of 40 Bobcoins

Every session in bob_sessions/

Exhibit 10: the numbers



Real repositories, real results

56

proven cases, none
written by hand

3

repositories, in Go
and Python

90s

from a new repository
to its first case

106

automated tests,
all passing

IBM/sarama (27 cases), IBM/python-sdk-core (28), google/uuid (1)



Why teams would use it

Zero authoring

Point it at a repository. Its history becomes the course.

Trustworthy

Every case is proven by the project's own tests. Nothing is guessed.

Private by design

Code and test runs stay on each developer's machine. Only PR text goes to IBM's AI.

Next

A team server: one shared case library, leaderboards, and a manager's view of who solved what.

Who pays, and why now

Buyer

Engineering managers and developer-enablement teams at companies with 50 to 5,000 engineers.

User

New hires and junior developers in their first 90 days on a codebase.

Channel

Open-source maintainers onboarding new contributors: free, public, and visible to every developer who visits.

36.5M

professional developers worldwide, up 70% in three years

SlashData, 2025

106K

US openings for developers and testers every year, each one a new person to onboard

US BLS, 2025-35

72%

of engineering leaders say new hires need over a month for their first 3 meaningful PRs

Cortex, 2024

50%

of programming time goes to finding and fixing bugs, the exact skill DejaBug trains

Cambridge Judge, 2013

Open source First, then a team plan

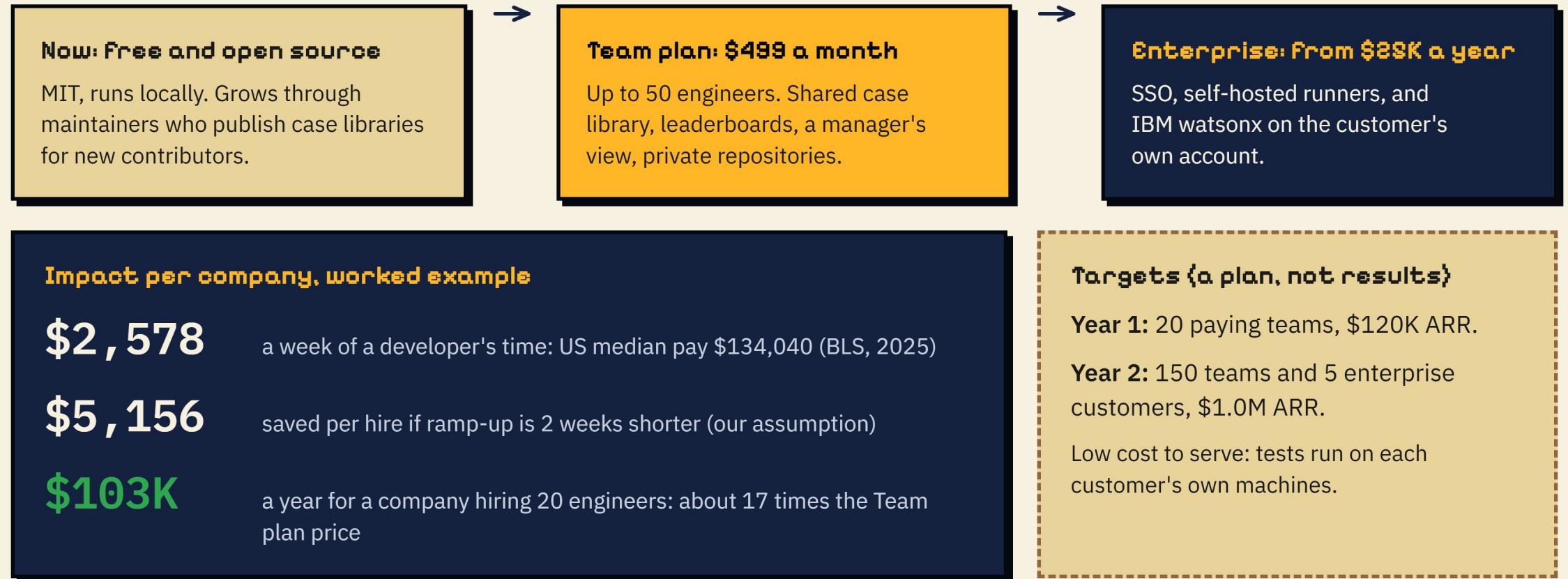


Exhibit 13: your turn

Your repository already has a training course hidden in its history.

Open invitation

terminal

```
$ git clone https://github.com/ahammadshawki8/DejaBug  
$ cd DejaBug && npm install  
$ npm run dev
```

Try it in your browser: ahammadshawki8.github.io/DejaBug

Open source, MIT

Debug the past. Level up the Future.

